ES 666 Computer Vision - Assignment 1

The Photographer's Problem

Assume you are a reputed photographer. To gain visibility and popularity, you built a website and uploaded few images.

Within a month a travel blog was running one of your photograph as a header image without any credit or attribution. When you complained, the reply was a shrug: *prove it's yours.* And you couldn't. A file proves nothing about who made it — anyone can hold a copy of a JPEG.

A naive first instinct is to stamp a logo across the corner of every image. You tried it and your colleagues hated it immediately. You wanted a clean image.

A nice idea would be a mark that is **there** but **not visible**. Something hidden into the image itself, and which you could later extract, in front of a judge, to say *this is mine, and here is the proof.*

Use your computer vision skills to solve the problem.

Key Idea

Get a QR code of your **Roll No.** from any QR code generator online. Store it (either as PNG/JPG/JPEG) in the root directory of this project and metion the path in `.env` file. Ensure the the QR code is square (or roughly square if you take screenshot). This is just for coding convenience.

You would be hiding the QR code in the image as watermark, which you can extract later.

Task 1

Watermarking

Assume a single channel of a cover (original) image, $C \in \mathbb{R}^{H \times W}$ and a QR-watermark, $Q \in \mathbb{R}^{k \times k}$ with pixels scaled to $[0, 1]$. The images can be decomposed via SVD as:

$$C = U \Sigma V^T, \qquad Q = U_q \Sigma_q V_q^T$$

> NOTE: The template code assumes $k = \min(H, W)$ and resizes the the QR image accordingly. For coding conveience, you can simply use the top k singular values and vectors for the task. Further description of the task use this assumption:
>
> 1. $U \in \mathbb{R}^{H \times k}$, $V^T \in \mathbb{R}^{k \times W}$
> 2. $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_k)$ where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k$
> 3. $U_q \in \mathbb{R}^{k \times k}$, $V^T \in \mathbb{R}^{k \times k}$
> 4. $\Sigma_q = \text{diag}(\sigma_1^q, \dots, \sigma_k^q)$ where $\sigma_1^q \geq \sigma_2^q \geq \cdots \geq \sigma_k^q$

Add the watermark's singular values into the cover's, scaled by the strength $\alpha > 0$:

$$\tilde{\sigma}_i = \sigma_i + \alpha \, \sigma_i^q, \qquad i = 1, \dots, k$$

$$\tilde{\Sigma} = \text{diag}(\tilde{\sigma}_1, \dots, \tilde{\sigma}_k)$$

Rebuild the image using the cover's **own** structure U and V^T with only singular values changed, so the photograph still looks like a nice photograph. You will have to clip the values of the resulting image so that `Matplotlib` displays it as it is.

$$C_\alpha = \text{clip}\big(U\tilde{\Sigma}V^T,\ 0,\ 1\big)$$

where the clipping keeps the result a displayable image:

$$\text{clip}(x, 0, 1) = \min\big(\max(x, 0),\ 1\big)$$

You keep three things private — they are his **key**:

$$\text{key} = \big(\Sigma,\ U_q,\ V_q^T\big)$$

that is, the original photo's singular values and the watermark's singular vectors. You then publishe C_α and nothing else.

Recovery

Now, when your photo turns up in the internet without any credits, you need a way to recover the QR code from the copy of the image. You download the copy, C'_α — the watermarked image, possibly damaged along the way by resaving or a filter. You can run the decomposition on it:

$$C'_\alpha = U'\Sigma'V'^T, \qquad \Sigma' = \text{diag}(\sigma'_1, \dots, \sigma'_k)$$

Subtract the original singular values you kept as the key, and divide by the strength α:

$$\tilde{\sigma}_i^q = \frac{\sigma'_i - \sigma_i}{\alpha}, \qquad i = 1, \dots, k$$

$$\tilde{\Sigma}_q = \text{diag}(\tilde{\sigma}_1^q, \dots, \tilde{\sigma}_k^q)$$

These are just the singular values. Reassembles these with the watermark's singular vectors from the key:

$$\tilde{Q} = \text{clip}\left(U_q\tilde{\Sigma}_q V_q^T,\ 0,\ 1\right)$$

This way you can recover the QR code.

If the blog's file were untouched and no clipping had occurred, the recovery would be exact.

Coding Questions (4 Marks)

Go through `task1.py` and write your code wherever you are asked to write the code. Don't modify any other section if not necessary. You will have to implement some functions and correctly call the functions in the code. Take care of the following points:

1. Implement SVD from scratch as discussed in class. You can use `numpy.linalg.eigh` for eigen value decomposition.
2. The watermark is already resized to $k \times k$ when passed to the functions. No need to resize again.
3. Images are already scaled to $[0, 1]$. No need to rescale again.

4. After successfully running `task1.py`, you will get results in `plots/task1_results.png`. The cover image after watermarking should look very similar to before watermarking. And a QR scanner must be able to extract your roll number from the recovered QR code. There may be some noise in the recovered QR code due to clipping operations.

> HINTS:
>
> 1. Ensure near zero (say $\leq 10e-9$) singular values are clipped to 0 so that they are not negative due to floating point behavior.
> 2. After decomposing matrix C into U, Σ and V, you *may have* to check if $C \cdot v_i$ and $\sigma_i u_i$ are not flipped (due to the behavior of `numpy.linalg.eigh`). If they are, flip u_i by multiplying by -1.

Subjective Questions

Write your answer in a file named `subjective_answers_task1.md`. Answer each question completely to get full marks on respective question. No partial marks.

1. Why using top k singular values and vector works for our tasks? Can we use less than top k singular values and vectors? If yes, what is the minimum value? If not, why? (0.5 marks).

2. Assume grayscale cover image of size $H \times W$, watermark of size $l \times l$ where $l < min(H, W)$ and number of singular values for watermark is less than that of the cover image. To which singular values of cover image will you add the singular values of the water mark and why? Options are (0.5 Marks):

   (a) Top l singular values of cover image

   (b) Least l singular values of cover image

   (c) All of the above

Task 2

In task 1, the value of α was hard coded. But in practical applications, there should be a way to decide the correct value of α. But, for deciding this there needs to be a metric to compare cover images with watermarked image and QR codes with the recovered QR codes. For this purpose, you would use **PSNR: Peak Signal-To-Noise Ratio** which compares an image I against a reference I_{ref} of the same size:

$$\text{MSE} = \frac{1}{N} \sum_{p=1}^{N} \left(I_{\text{ref}}(p) - I(p) \right)^2$$

$$\text{PSNR} = 20 \log_{10} \left(\frac{\text{MAX}}{\sqrt{\text{MSE}}} \right)$$

Symbol	Meaning
N	Total number of values compared (height × width × 3 channels)
p	Index over every pixel and channel

Symbol	Meaning
MAX = 1	Largest possible pixel value, since images are scaled to $[0, 1]$

Higher PSNR means the two images are more alike. There are two important things to care about:

1. **Invisibility** — PSNR(C, C_α), how well the watermarked photo matches the original.
2. **Recoverability** — PSNR$(Q, \tilde{Q})$, how faithfully the mark comes back.

Raising α improves neither for free: it pushes the first down while, past a point, clipping drags the second down as well. Sweeping α and plotting both curves is how one can pick the best value for it

Coding Questions (1.5 Marks)

Go through `task2.py` and write your code wherever you are asked to write the code. Don't modify any other section if not necessary. You will have to implement some functions and correctly call the functions in the code. After successfully running `task2.py`, you will get results in `plots/task2_psnr_results.png`. There should be two plots in the resultant image.

Subjective Questions

Write your answer in a file named `subjective_answers_task2.md`. Answer each question completely to get full marks on respective question. No partial marks.

1. What do you think PSNR is measuring? What could be a disadvantage of it? (0.25 Marks)
2. Briefly explain, what does the script `task2.py` is doing? (0.25 Marks)

Task 3

A photograph on the internet does not stay pristine. You need to check how Gaussian Noise impacts the recovery of the QR code. Gaussin Noise can be added as:

$$C'_\alpha = \text{clip}(C_\alpha + \eta,\ 0,\ 1), \qquad \eta(p) \sim \mathcal{N}(0, s)$$

where η is the noise and s its standard deviation. Higher s would lead to more noise leading to difficulty in recovery. You need to find best α for various values for s.

Coding question (1.5 Marks)

Go through `task3.py` and write your code wherever you are asked to write the code. Don't modify any other section if not necessary. You will have to implement some functions and correctly call the functions in the code. After running `task3.py`, you will get following results in `plots` directory:

- task3_psnr_vs_alpha.png : PSNR plots
- task3_watermark_results_stddev_0.01.png : Noisy Watermarked image for given stddev and recovered QR for each cover image and multiple α
- task3_watermark_results_stddev_0.05.png : Noisy Watermarked image for given stddev and recovered QR for each cover image and multiple α
- task3_watermark_results_stddev_0.15.png : Noisy Watermarked image for given stddev and recovered QR for each cover image and multiple α
- task3_watermark_results_stddev_0.1.png : Noisy Watermarked image for given stddev and recovered QR for each cover image and multiple α

Subjective Questions

Write your answer in a file named `subjective_answers_task3.md`. Answer each question completely to get full marks on respective question. No partial marks.

1. For each image and each stddev, write the minimum value of α for which a QR scanner is able to extract your roll number? In which case the QR scanner failed to extract the roll no.? (0.25 Marks)
2. Write key obervations from the plots in `task3_psnr_vs_alpha.png`? (0.25 Marks)

Task 4

Every site that reposts an image re-encodes it. Multiple resaving and re-encoding via multiple encoding/compression algorithms can lead to loss of information. Here, we need to check the impact of JPEG compression on the recovery the QR code. When an image is saved via JPEG compression, it uses something called quality factor, $qf \in [0, 100]$. Lower qf discards more detail leading to difficulty in recovery. You need to find best α for various values for qf.

Coding questions (0.5 Mark)

Go through `task4.py` and write your code wherever you are asked to write the code. Don't modify any other section if not necessary. Function for JPEG compression is already implemented. You have to correctly call the functions in the code. After running `task4.py`, you will get following results in `plots` directory:

- task4_psnr_vs_alpha.png: PSNR plots
- task4_watermark_results_quality_30.png : Compressed Watermarked image for given quality and recovered QR for each cover image and multiple α
- task4_watermark_results_quality_50.png : Compressed Watermarked image for given quality and recovered QR for each cover image and multiple α
- task4_watermark_results_quality_70.png : Compressed Watermarked image for given quality and recovered QR for each cover image and multiple α
- task4_watermark_results_quality_90.png : Compressed Watermarked image for given quality and recovered QR for each cover image and multiple α

Subjective Questions

Write your answer in a file named `subjective_answers_task4.md`. Answer each question completely to get full marks on respective question. No partial marks.

1. For each image and each quality factor qf, write the minimum value of α for which a QR scanner is able to extract your roll number? (0.25 Marks)
2. Write key obervations from the plots in `task4_psnr_vs_alpha.png`? (0.25 Marks)